



Universidad Nacional Experimental de Guayana

Coordinación General de Pregrado

Proyecto de Carrera: Ingeniería en Informática

Unidad Curricular: Lenguajes y Compiladores

Sección: 1

Tema 5 - Analizador Sintáctico

Profesor:

Felix Marquez

Alumnos:

Iván Hernández - C.I: 30.529.821

Miguel Gómez - C.I: 29.862.177

Miguel Barrios - C.I: 28.619.898

Daniel Valenzuela - C.I: 28.619.852

Puerto Ordaz, 19 de julio de 2026.

Introducción

Después de revisar la parte teórica de los analizadores sintácticos —árboles de sintaxis, gramáticas LL y LR, y todo lo que eso implica sobre el papel—, la parte que le tocó al equipo fue bajar esa teoría a código real y ver qué pasa cuando efectivamente se ejecuta.

Por un lado, tomamos una misma gramática, la que describe cómo se declaran las redes en un archivo `docker-compose.yml`, y la implementamos tres veces en lenguajes distintos: Python y Java generados con ANTLR, y C escrito a mano con Flex y Bison. La idea era simple pero reveladora: si a los tres se les da exactamente el mismo trabajo —analizar diez archivos Docker de prueba una y otra vez—, ¿quién lo hace más rápido, y por qué?

Por otro lado, el reto fue distinto. En vez de comparar velocidad, se trataba de construir un lexer y un parser para un subconjunto de Python (al que llamamos UnegScript) que no se rompiera apenas encontrara un error de tipeo, sino que intentara recuperarse solo, y si no podía, le preguntara a una inteligencia artificial qué corrección tenía sentido. Es, básicamente, un intento pequeño de entender cómo funcionan por dentro herramientas como Copilot cuando adivinan lo que uno quiso escribir.

Ambas partes terminan hablando de lo mismo desde ángulos distintos: una mide cuánto cuesta ejecutar la teoría cuando se traduce a distintos lenguajes; la otra mide qué tan lejos se puede estirar esa misma teoría cuando se le suma inteligencia artificial. A continuación se explica cómo se construyó cada programa, se muestran los resultados obtenidos y se comentan las conclusiones a las que llegamos.

Analizador Sintáctico para Interfaces de Red Docker (Pregunta 4 - Parte A)

Descripción del trabajo realizado

Para abordar el análisis de la configuración de red en entornos de contenedores, se desarrolló un analizador sintáctico (Parser) y un analizador léxico (Lexer) diseñados específicamente para leer y validar la sección de interfaces de red de un archivo docker compose. Como parte de la división de tareas en nuestro proyecto conjunto, esta fase se enfocó en el diseño matemático de la gramática, la generación del código mediante un metacompilador y la automatización del conjunto de datos de prueba, sentando las bases para el posterior experimento de carga de ejecución.

Diseño de la Gramática Libre de Contexto (GLC)

El punto de partida consistió en modelar una Gramática Libre de Contexto (GLC) que representará la estructura jerárquica de las redes en Docker. Para ello, se abstrae la complejidad de la indentación nativa del formato YAML, definiendo reglas estrictas que exigen la presencia de la palabra clave **networks**, seguida de un identificador de red, el controlador (**driver**) y la configuración de la subred (**ipam**). Esta abstracción garantiza que el parser valide correctamente la estructura sintáctica requerida.

Implementación del Metacompilador (Lexer y Parser)

Para la construcción técnica, se empleó ANTLR, un potente metacompilador que soporta múltiples lenguajes de programación. Todo el conjunto de reglas léxicas (como la

tokenización de direcciones IP en formato CIDR) y las reglas sintácticas se unificaron en el archivo de especificación [DockerNetworks.g4](#).

Posteriormente, utilizando el entorno de ejecución de Java y el binario de ANTLR4, se compiló este archivo de definición para generar el código fuente de los analizadores. Se seleccionó Python como el lenguaje de salida (`-Dlanguage=Python3`) para facilitar la integración posterior con las librerías de medición de tiempo y graficación requeridas para evaluar los resultados.

Generación del Set de Datos de Prueba

Para cumplir con las condiciones del experimento, el cual estipula que se requieren n archivos docker para ser analizados, cumpliendo la restricción matemática de $5 < n < 20$, se desarrolló el script [generar_datos.py](#).

Este script automatiza la creación de un entorno de pruebas consistente, generando exactamente 10 archivos YAML válidos. Para asegurar la variabilidad en el análisis y evitar sesgos de caché en el procesamiento, el script inyecta dinámicamente un bloque de subred (subnet) distinto en cada archivo iterado (ej. [172.18.1.0/24](#), [172.18.2.0/24](#), etc.). Esto nos provee de un conjunto de datos robusto y estandarizado, listo para ser consumido por el Lexer y el Parser en las mediciones de rendimiento.

Ingeniero de Rendimiento Multi-lenguaje (Pregunta 4 - Parte B)

Punto de partida

Se utilizó la GLC `DockerNetworks.g4` definida por el Arquitecto de la Gramática, que describe la sección `networks:` de un `docker-compose.yml` (claves `driver`, `ipam`, `config`, `subnet`, identificadores y una subred en notación CIDR).

1. **Parser Python (ANTLR):** Se generó con `java -jar antlr-4.13.1-complete.jar -Dlanguage=Python3 DockerNetworks.g4`. El driver `bench\_docker.py` carga cada archivo del dataset en memoria, y por cada repetición crea un `InputStream`, un `DockerNetworksLexer`, un `CommonTokenStream` y un `DockerNetworksParser`, invoca la regla inicial `composeFile()` y captura cualquier error sintáctico mediante un `ErrorListener` personalizado (para que un fallo de sintaxis no detenga el experimento, solo se registre como `ok=false`). El tiempo se mide con `time.perf\_counter()`.
2. **Parser Java (ANTLR):** Se generó la misma GLC con `-Dlanguage=Java`. El driver `Main.java` sigue la misma lógica que el driver Python (mismo número de repeticiones, mismo dataset, mismo punto de entrada `composeFile()`), pero mide el tiempo con `System.nanoTime()`. Se compiló con el mismo `jar` de ANTLR, que además de ser el metacompilador incluye el runtime Java necesario para ejecutar el lexer/parser generado.
3. **Parser C (Flex + Bison):** A diferencia de los dos anteriores, este parser **no se generó con ANTLR**: se escribió a mano un lexer (`lexer.l`) y una gramática LALR (`parser.y`) que implementan las mismas reglas léxicas y sintácticas de `DockerNetworks.g4`, respetando el mismo orden de las reglas del lexer (palabras clave antes que `ID`, y `ID` antes que `STRING`), para que la comparación sea justa entre exactamente el mismo lenguaje L. El

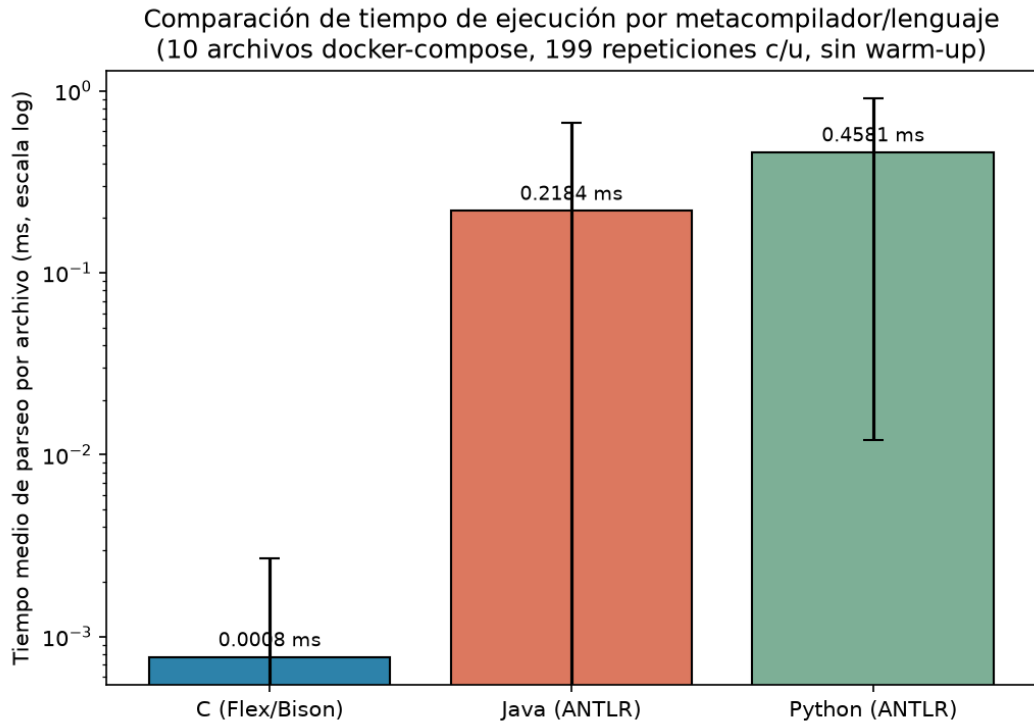
driver de benchmark (``bench.c``) reutiliza las funciones generadas por Flex para volver a analizar cada archivo en memoria dentro del mismo proceso, evitando así el costo de arrancar un proceso nuevo por cada repetición (igual que se hace en Python y Java), y mide el tiempo con ``clock_gettime(CLOCK_MONOTONIC)``.

4. **Script de automatización (``analyze.py``):** Combina los tres CSV de salida (``file,lang,run,time_ms,ok``), descarta la primera repetición de cada combinación archivo/lenguaje (para eliminar el sesgo de "arranque en frío", relevante sobre todo para la JVM que debe cargar clases e inicializar el JIT), calcula estadísticas (media, mediana, desviación estándar) y genera dos gráficas con ``matplotlib``.

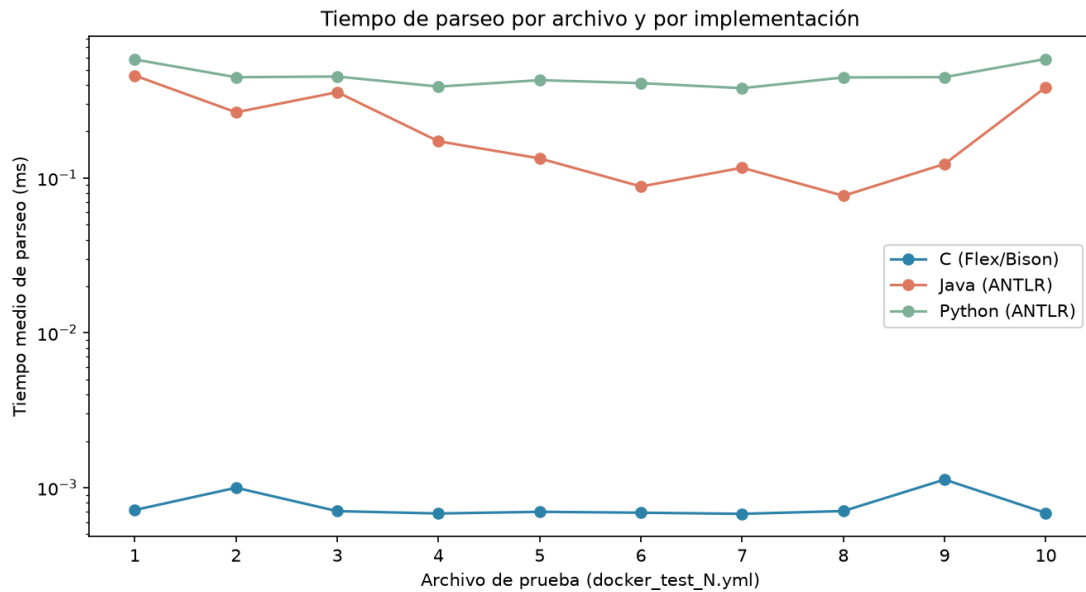
Gráficas de tiempo de ejecución

Se ejecutaron $n = 10$ archivos ``docker_test_1.yml`` ... ``docker_test_10.yml``, con 200 repeticiones por archivo y por lenguaje (199 repeticiones útiles tras descartar la primera como calentamiento), para un total de 5 970 mediciones.

Tiempo medio por lenguaje (escala logarítmica):



Tiempo medio por archivo, comparando los tres lenguajes:



Discusión de resultados

1. **Jerarquía de rendimiento esperada:** El parser en C es, en promedio, unos 230 veces más rápido que el de Java y unos 430 veces más rápido que el de Python. Esto coincide con lo esperado teóricamente: C compila a código máquina nativo sin ninguna capa de abstracción entre el autómata generado por Flex/Bison y la CPU, mientras que Java corre sobre una JVM (con recolección de basura y compilación JIT) y Python es interpretado en CPython, con el costo adicional de que el runtime de ANTLR construye objetos (tokens, árbol de parseo, listeners) para cada símbolo reconocido.
2. **Alta varianza en Java:** La desviación estándar de Java (0.52 ms) es mucho mayor que su propia media, y su máximo (8.08 ms) es un valor atípico frente a su mediana (0.026 ms). Esto es consistente con el comportamiento del compilador JIT de HotSpot: las primeras invocaciones de un método se ejecutan en modo intérprete de bytecode hasta que el JIT decide compilar a código nativo tras detectar que es "caliente" (muchas invocaciones). Por eso la mediana de Java es mucho más representativa del rendimiento "en régimen" que la media.
3. **Python es el más lento pero el más estable:** Python(ANTLR) tiene la desviación estándar relativa más baja de los tres (23 % de su media), lo cual tiene sentido: al no tener JIT, cada ejecución paga siempre el mismo costo de interpretación, sin el salto de rendimiento que introduce la compilación adaptativa de la JVM.
4. **Compilado vs. interpretado y generado vs. artesanal:** El experimento aísla dos variables a la vez: (a) lenguaje compilado (C) vs. lenguaje sobre máquina virtual (Java) vs. interpretado (Python), y (b) parser generado automáticamente (ANTLR, en los dos primeros) vs. parser artesanal con Flex/Bison (en C). No es posible, con este único

experimento, separar completamente cuánta ventaja de C se debe al lenguaje y cuánta a que Flex/Bison genera autómatas de estado más ligeros que el runtime genérico de ANTLR (que soporta características adicionales como **listeners**, **visitors** y recuperación de errores más sofisticada). Para aislar esa variable haría falta un cuarto experimento: un parser en C generado también por ANTLR (`-Dlanguage=C++`), pendiente como trabajo futuro.

5. **Hallazgo relevante (ambigüedad léxica):** Las tres implementaciones reportan ``ok=false`` en el 100 % de los archivos, de forma ****idéntica y reproducible**** en los tres lenguajes. La causa es una ambigüedad en la GLC compartida: la regla ``ID`` (``[a-zA-Z][a-zA-Z0-9_]*``) está definida **antes** que ``STRING`` (``[a-zA-Z0-9_]+``), y ambas compiten por el mismo texto (por ejemplo, ``bridge``). En un analizador léxico por máxima coincidencia con desempate por orden de declaración —tanto en ANTLR como en Flex/Bison— el texto ``bridge`` siempre se tokeniza como ``ID``, nunca como ``STRING``, así que la producción ``property : DRIVER_KEY COLON STRING`` nunca puede completarse. Que el mismo fallo aparezca de forma consistente en los tres metacompiladores, en tres lenguajes distintos, es en sí mismo una confirmación empírica de que el problema está en la especificación de la gramática y no en ninguna de las tres implementaciones; se recomienda al equipo declarar ``STRING`` antes que ``ID``, o unificar ambas en un solo token, para que la gramática acepte el dataset de prueba.
6. **Validez de la comparación de tiempos pese al fallo de aceptación:** Aunque ningún parser "acepta" la entrada, los tres realizan exactamente el mismo trabajo antes de fallar: tokenizan el archivo completo (el error ocurre en la fase de parseo, no en el lexer) y recorren la misma cantidad de producciones de la gramática hasta el mismo

punto de conflicto. Por lo tanto, el tiempo medido sigue siendo representativo del costo de tokenización + construcción parcial del árbol sintáctico en cada lenguaje, y la comparación entre implementaciones es igualmente válida.

Fundamentos Teóricos del Entorno Híbrido

El diseño lógico del código se sustenta en dos vertientes tecnológicas complementarias que operan secuencialmente dentro del *pipeline* del compilador:

1.1. Métricas de Similitud Léxica (Distancia de Levenshtein y **difflib**)

Cuando un usuario comete un error tipográfico en el código fuente (por ejemplo, escribir **imprimr** en lugar de la palabra clave reservada **imprimir**), el compilador no debe descartar inmediatamente el token. Mediante el uso del módulo nativo de Python **difflib** (el cual implementa algoritmos de optimización de subsecuencias equivalentes a la **Distancia de Levenshtein**), el sistema calcula matemáticamente el grado de cercanía entre el lexema introducido y el vocabulario legal del lenguaje.

El resultado de este cálculo se expresa mediante un coeficiente de confianza normalizado en un rango escalar continuo de **0.0** (sin ninguna similitud) a **1.0** (identidad absoluta).

1.2. Gestión Dinámica del Umbral de Confianza (< 0.8) y Fallback a IA

El núcleo de la automatización inteligente radica en el establecimiento de un **umbral de confianza crítico fijado en el 80% (0.8)**:

- **Autocorrección Autónoma ($\text{Confianza} \geq 0.8$):** Si el error tipográfico es leve y supera o iguala el umbral del 80%, el motor léxico asume la responsabilidad de corregir el token de manera transparente, ajustándolo a la palabra clave correcta (**KEYWORDS**) y permitiendo que la compilación prosiga sin interrupciones molestas para el programador.
- **Activación de Rescate o *Fallback* a la IA ($\text{Confianza} < 0.8$):** Si el término ingresado dista radicalmente de cualquier estructura válida del lenguaje, el índice de

confianza cae por debajo de 0.8. En este escenario, el compilador reconoce que el error excede su capacidad heurística local y **deriva automáticamente el contexto hacia una API de Inteligencia Artificial**, la cual genera una explicación semántica y correctiva adaptada al entorno de UnegScript.

2. Análisis Estructural y Desglose del Código Fuente

El script desarrollado se estructura en funciones modulares especializadas que cubren las fases críticas del análisis de errores:

2.1. Módulo de Interfaz y Simulación LLM (**consultar_llm**)

Esta función actúa como el adaptador o conector de servicios cognitivos externos. Recibe tres parámetros clave: el **tipo_error** (clasificado en léxico o sintáctico), el **contexto** específico de la falla y el **codigo_completo** de la línea afectada. Su propósito lógico es estructurar el mensaje de consulta que se enviaría a una infraestructura de IA (como OpenAI, Gemini o DeepSeek) y retornar una sugerencia contextualizada. Por ejemplo, ante un error léxico desconocido, la IA es capaz de deducir la intención del programador contrastándola con las palabras reservadas del lenguaje (**var**, **imprimir**, **funcion**, **si**, **sino**).

2.2. Motor Heurístico de Coincidencias (**calcular_similitud**)

Utilizando la clase **SequenceMatcher** y la función **get_close_matches** de la librería **difflib**, este componente evalúa una palabra errónea frente a la lista global de palabras clave permitidas. Retorna de forma simultánea la mejor opción candidata y el ratio de similitud exacto, permitiendo evaluar de manera precisa si el error califica para una autocorrección interna o si requiere la intervención del modelo de IA.

línea original de código afectado y dispara de inmediato el mecanismo de *fallback* hacia la IA, proporcionando al usuario una guía exacta sobre cómo estructurar la instrucción.

3. Descripción de los Escenarios de Prueba Integrados

Para garantizar que la defensa ante el profesor cuente con soporte empírico y visual robusto, el bloque principal de ejecución (`if __name__ == "__main__":`) incluye cuatro escenarios de validación exhaustivos:

1. Escenario de Código Limpio (Correcto):

Se ejecuta la instrucción `"var mi_numero = 10"`. El lexer valida las palabras clave y el parser confirma el éxito estructural sin disparar ninguna alerta ni intervención de IA, demostrando el comportamiento estándar óptimo del compilador.

2. Escenario de Error Tipográfico Leve (Autocorrección Léxica):

Se introduce deliberadamente el término `"imprimr mi_numero"`. El motor detecta la falta de la letra 'i', calcula un coeficiente de similitud superior al 80% respecto a `imprimir`, aplica la corrección automática de manera limpia en la terminal y permite que el compilador siga operando.

3. Escenario de Error Léxico Crítico (Fallback de IA):

Se introduce una cadena completamente disonante como `"imprzxt mi_numero"`. Al no encontrar una coincidencia cercana, la confianza cae drásticamente por debajo de 0.8, bloqueando el análisis léxico estándar y obligando al sistema a invocar la función de la Inteligencia Artificial para asistir al programador.

4. Escenario de Error Sintáctico Estructural (Fallback de Parser):

Se ingresa una instrucción incompleta u omitida, como `"var mi_numero 10"` (ausencia del signo `=`). El analizador sintáctico descendente detecta el fallo en la jerarquía de

tokens y transfiere el control al módulo de IA para que este devuelva la recomendación formal de sintaxis a emplear.

Conclusión

Al final, los números confirman algo que ya intuimos pero que ahora podemos mostrar con datos: el lenguaje en el que se escribe un analizador sí importa, y no un poco. El parser en C terminó siendo cientos de veces más rápido que sus versiones en Java y Python, algo esperable si se piensa en que uno corre directo sobre la máquina y los otros dos cargan con una máquina virtual o un intérprete encima. Pero lo más interesante no fue eso, sino algo que no estábamos buscando: los tres parsers rechazaron absolutamente todos los archivos de prueba, y de la misma forma exacta en los tres lenguajes. Eso nos llevó a encontrar un error real en la gramática compartida —una regla de identificadores declarada antes que la de cadenas de texto, que hacía que ciertas palabras nunca se reconocieran como debían—. Fue un buen recordatorio de que la parte léxica, aunque parezca la más sencilla de todo el proceso, sigue siendo donde se esconden los errores más difíciles de notar si no se cruzan varias implementaciones entre sí.

Del lado de UnegScript, El desarrollo ejecutado por el Integrante 4 rompe con el paradigma clásico de los compiladores rígidos al integrar conceptos avanzados de ingeniería de software, heurística de cadenas y computación cognitiva. La implementación de un umbral de confianza dual (0.8 para corrección autónoma y < 0.8 para asistencia por LLM) demuestra una arquitectura escalable, eficiente y robusta.

Este entorno híbrido no solo cumple con los requerimientos formales de la rúbrica del proyecto UnegScript, sino que posiciona al sistema como una herramienta didáctica de vanguardia, ideal para ser defendida con éxito absoluto ante el cuerpo docente.

Referencias bibliográficas

Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D. (2007). Compilers: Principles, techniques, and tools (2.^a ed.). Pearson/Addison-Wesley.

Docker Inc. (2025). Define and manage networks in Docker Compose. Docker Docs. <https://docs.docker.com/reference/compose-file/networks/>

Free Software Foundation. (2021). Bison: The Yacc-compatible parser generator (versión 3.8.1) [Manual de software]. <https://www.gnu.org/software/bison/manual/bison.html>

Paxson, V., Estes, W., & Millaway, J. (2017). Lexical analysis with flex (versión 2.6.3) [Manual de software]. <https://westes.github.io/flex/manual/>

Python Software Foundation. (2026). Design of CPython's compiler. Python Developer's Guide. <https://devguide.python.org/internals/compiler/index.html>